Projekt: Browary Bilba – Algorytmy Knutha-Morrisa-Pratta i Boyera-Moore'a, kompresja i dekompresja na podstawie kodów Huffmana

Data testów: 04.06.2025
Autor testów: Michał Urbańczyk

Środowisko testowe:

Język programowania: C++

Platforma: Linux / Windows

Kompilator: g++

Biblioteki: "doctest.h"

Framework testowy: doctest v2

Celem raportu jest przedstawienie wyników testów jednostkowych dla klas: KnuthMorrisPratt, BoyerMoore, Huffman oraz Dekoder. Testy mają na celu weryfikację poprawności wyników tablic wyszukiwania oraz sprawdzenia czy kompresja i dekompresja nie zmienia tekstu pierwotnego.

Zakres testów:

1) KnuthMorrisPratt i BoyerMoore

- Poprawna tablica wynikowa zawierająca informację o wystąpieniach wzorca w tekście (w przypadku braku wystąpień zwraca pustą tablice)

2) Huffman i Dekoder

- Porównanie tekstu po skompresowaniu go do pliku i odkodowaniu go z powrotem w formie tekstu

Wykonane testy:

1) KnuthMorrisPratt i BoyerMoore (?? - odpowiednio KMP lub BM)

Nazwa testu	Opis
"??: Wyszukiwanie wzorca 1 elementowego"	Testuje wyszukiwanie wzorca składającego się z jednego znaku
"??: Czy rozoznia male i duze litery"	Testuje rozróżnianie dużych i małych liter przez wyszukiwarkę

“??: Wzorzec wystepuje na poczatku tekstu”	Testuję, czy zostanie znaleziony wzorzec, jeżeli znajduje się na początku tekstu
“??: Wzorzec wystepuje na koncu tekstu”	Testuję, czy zostanie znaleziony wzorzec, jeżeli znajduje się na końcu tekstu
"??: Wzorzec ktory nachodzi na siebie w tekscie"	Testuje, czy zostaną znalezione wszystkie występowania wzorca, jeżeli występowania nachodzą na siebie w tekście
"??: Wzorzec dlugosci tekstu"	Testuje wyszukanie wzorca o tej samej długości co tekst
"??: Pusty wzorzec"	Testuje czy przy pustym wzorcu zostanie znalezione nic
"??: Wzorzec dluzszy niz tekst"	Testuje czy przy wzorcu dłuższym od tekstu zostanie znalezione nic

2) Huffman i Dekoder:

Nazwa testu	Opis
"Kompresja i dekompresja prostego tekstu"	Testuje czy prosty tekst zostanie bez zmian taki sam po jego kompresji i dekompresji
"Kompresja i dekompresja zlozonego tekstu"	Testuje czy większy tekst zawierający białe znaki (“ ”, ”\n”, “\t”) zostanie bez zmian taki sam po jego kompresji i dekompresji
"Kompresja i dekompresja pustego tekstu"	Testuje czy próba kompresji pustego tekstu utworzy poprawnie plik do dekompresji, która zwróci pusty tekst (nie zatrzyma niespodziewanie programu)
"Kompresja i dekompresja tekstu z powtórzeniami"	Testuje kompresje i dekompresje tekstu zawierającego powtórzenia znaków obok siebie
"Kompresja tekstu z jednym znakiem"	Testuje przypadek, w którym mamy w tekście tylko jeden różny znak np: “aaaa”
"Kompresja tekstu z unikalnymi znakami"	Testuje tekst zawierający różne znaki z tablicy ASCII

Wyniki testów dla wyszukiwarek:

Testy przeprowadzone – 22

Testy zakończone sukcesem – 22

Testy zakończone niepowodzeniem – 0

Przeprowadzone asercje – 22

Asercje zakończone sukcesem – 22

Asercje zakończone niepowodzeniem – 0

Wykryte defekty: Brak

Pokrycie funkcjonalne: Wysokie (na podstawie analizy testowanych funkcjonalności)

Wyniki testów dla kompresji i dekompresji:

Testy przeprowadzone – 6

Testy zakończone sukcesem – 6

Testy zakończone niepowodzeniem – 0

Przeprowadzone asercje – 12

Asercje zakończone sukcesem – 12

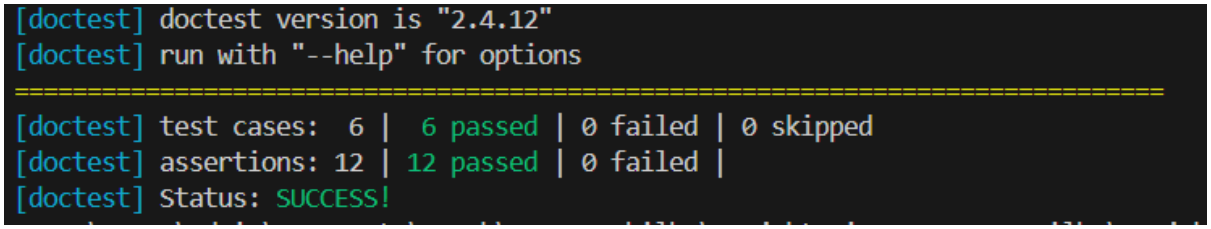
Asercje zakończone niepowodzeniem – 0

Wykryte defekty: Brak

Pokrycie funkcjonalne: Wysokie (na podstawie analizy testowanych funkcjonalności)

Wynik z konsoli:

```
[doctest] doctest version is "2.4.12"
[doctest] run with "--help" for options
===============================================================================
[doctest] test cases: 22 | 22 passed | 0 failed | 0 skipped
[doctest] assertions: 22 | 22 passed | 0 failed |
[doctest] Status: SUCCESS!
```



Podsumowanie

Przeprowadzone testy potwierdziły poprawność działania algorytmów Knutha-Morrisa-Pratta i Boyera Moore'a użytych do wyszukiwania oraz kodów Huffmana użytych do kompresji i dekompresji. Algorytmy prawidłowo znajdują występowanie wzorca w tekście, a kompresja i dekompresja tekstu działa poprawnie. Wszystkie scenariusze testowe zakończyły się sukcesem, a otrzymane wyniki były zgodne z oczekiwaniami. Implementacja działa stabilnie i spełnia założone wymagania funkcjonalne.